

Making large language models reliable data science programming copilots for biomedical research

Received: 11 October 2024

Accepted: 16 November 2025

Published online: 22 January 2026

 Check for updates

Zifeng Wang^{1,2,6}, Benjamin Danek^{1,2,6}, Ziwei Yang³, Zheng Chen⁴ & Jimeng Sun^{1,2,5} 

Large language models (LLMs) can generate impressive data visualizations from simple requests, yet their accuracy remains underexplored. Here we present a benchmark of 293 coding tasks derived from 39 studies across 7 biomedical research areas, including biomarkers, integrative analysis, genomic profiling, molecular characterization, therapeutic response, translational research and pan-cancer analysis. Benchmarking eight proprietary and eight open-source LLMs under various prompting strategies reveals an overall accuracy below 40%. This low accuracy raises serious concerns about the risk of propagating incorrect scientific findings when blindly relying on AI-generated analyses. Therefore, we develop an AI agent that begins with and iteratively refines an analysis plan before generating code, achieving 74% accuracy. We embody this insight in a platform that enables users to codevelop analysis plans with LLMs and execute them within an integrated environment. In a user study with five medical researchers, the platform enabled users to complete over 80% of the analysis code for three studies.

Data science plays a pivotal role in biomedical research¹. Completing data science projects for biomedicine requires years of coding expertise and a deep understanding of complex medical data while collaborating closely with medical researchers². Recently, the rising demand for data science skills and the limited supply of experienced practitioners have become serious bottlenecks³. Given this, developing methods to streamline the coding workflow not only enhances the productivity of data scientists but also empowers medical experts to directly engage in data analysis, unlocking the full potential of artificial intelligence (AI) to accelerate drug development and improve patient outcomes.

Large language models (LLMs) have been widely adopted to programming tasks⁴. Continuous efforts have been made to develop more powerful code-specific LLMs^{5–7}, refine prompting strategies⁸, integrate external knowledge through retrieval-augmented generation (RAG)^{9,10}

and enable LLMs to self-reflection¹¹. By far, LLMs have been extensively used and evaluated for general programming tests^{4,12–14}, software engineering¹⁵ and data analysis^{16,17}. In medicine, recent work has introduced LLMs to automate machine learning modelling¹⁸, support bioinformatic tool development¹⁹ and validate hypotheses^{20,21}. Nonetheless, few studies have systematically benchmarked the code accuracy of LLM-generated data science analyses in biomedical contexts. Moreover, there has been limited discussion on how data science code generation differs from general programming tasks, and how LLMs can be adapted to support domain experts effectively in this setting.

This study introduces a biomedical data science code benchmark of 293 analysis questions derived from 39 published biomedical studies and their associated patient-level datasets, named BioDSBench (Fig. 1a). The selected studies span key areas of biomedical research,

¹Keiji AI, Seattle, WA, USA. ²School of Computing and Data Science, University of Illinois Urbana-Champaign, Urbana, IL, USA. ³Bioinformatics Center, Institute for Chemical Research, Kyoto University, Kyoto, Japan. ⁴Institute of Scientific and Industrial Research, Osaka University, Osaka, Japan. ⁵Carle Illinois College of Medicine, University of Illinois Urbana-Champaign, Urbana, IL, USA. ⁶These authors contributed equally: Zifeng Wang, Benjamin Danek.  e-mail: jimeng@illinois.edu

including biomarker discovery, integrative analysis, genomic profiling, molecular characterization, therapeutic response, translational studies and pan-cancer analysis. We extracted and categorized analyses from these studies into eight representative types: gene expression, genomic alteration, survival analysis, clinical feature engineering, treatment responses, descriptive statistics, enrichment and pathway analysis, and data integration. The final question set was stratified across both study and analysis types (Fig. 1b,c). Each question is grounded in a figure or table from the original study, enabling expert data scientists to implement verified reference codes and test cases by comparing outputs with the reported results.

Benchmarking 16 LLMs with various adaptation strategies on BioDSBench leads to a clear conclusion: while performance differs across models, none is yet capable of reliably automating biomedical data science. The best-performing models achieve code accuracies below 70%, 45% and 32% on easy, medium and hard tasks, respectively (Fig. 2). Our case study further reveals a critical limitation that LLMs struggle to resolve the open-ended nature of data analysis requests, introducing substantial variability in how they interpret task intent and underlying dataset schemas. As a result, AI-generated analysis results may contain subtle errors that risk propagating misleading scientific conclusions. Even more concerning, domain experts often find it challenging to interpret or correct the generated code, limiting the trustworthiness and practical adoption of these tools. We release our benchmark dataset to support future research and improvement of AI agents in biomedical data science.

A deeper analysis demonstrates that an analysis plan-centric approach holds strong potential for bridging this gap. With guidance from structured plans, AI agents achieve accuracies of 90%, 76% and 55% on easy, medium and hard tasks, respectively. Crucially, these plans are interpretable and modifiable by domain experts, fostering productive human–AI collaboration in biomedical data science. To embody this insight, we developed a platform called DSWizard. A preliminary user study with five medical researchers indicates that this framework substantially improves the efficiency of biomedical data analysis. The platform is available for research use upon request at ref. 22.

Results

Creating data science tasks from biomedical publications

BioDSBench was designed to capture the real-world challenges in biomedical research. We selected multimodal patient-level datasets from cBioPortal, along with their associated published studies, as the data source²³. As shown in Fig. 1a, each study and its original patient data are publicly accessible. These datasets typically consist of three to ten relational tables per study, encompassing a wide range of clinical variables, such as demographics and survival outcomes, as well as molecular profiles, including gene expression, mutations, structural variants and copy number alterations. The tables vary widely in structure and can include tens of thousands of columns. This diversity allows us to create realistic data science questions grounded in complex, real-world data and to reproduce key findings reported in the original biomedical studies.

We stratified and sampled 39 studies that provide access to original patient-level data, covering seven major areas of biomedical research, including biomarker discovery, integrative analysis, genomics and others (Fig. 1b). The study categorization and selection process is detailed in Methods. From these studies, we identified and summarized the data science tasks performed, organizing them into eight analysis classes. We then extracted published results associated with these classes, resulting in a curated set of 293 analysis tasks spanning categories such as gene expression, genomic alteration, and enrichment and pathway analysis (Fig. 1c). The extraction and categorization procedures are also described in Methods. A list of example questions for each class is presented in Supplementary Note 2.

Data analysis questions and answers were manually developed on the basis of the selected analyses. Expert data scientists created the reference code for each question by reproducing results from a specific figure or table in the original study, along with corresponding test cases for automated evaluation (Fig. 2a; see details in Methods). In addition, they documented the reference solution as a high-level natural language analysis plan, which is used to evaluate the effectiveness of plan-driven prompting. Each coding task in the benchmark includes: (1) the raw question, (2) an input analysis plan, (3) a dataset schema description, (4) prerequisite code (referred to as the ‘prefix’), (5) a reference solution and (6) test cases. In experiments, the prefix code is prepended to the code generated and executed together. This set-up simulates a multistep analysis in a computational notebook environment, allowing us to assess how well LLMs perform when building on prior code. We further categorized the questions into easy, medium and hard levels based on the implementation complexity of the reference code (Methods), as illustrated in Fig. 1e. The pie charts in Fig. 1b summarize the external packages used across the reference solutions.

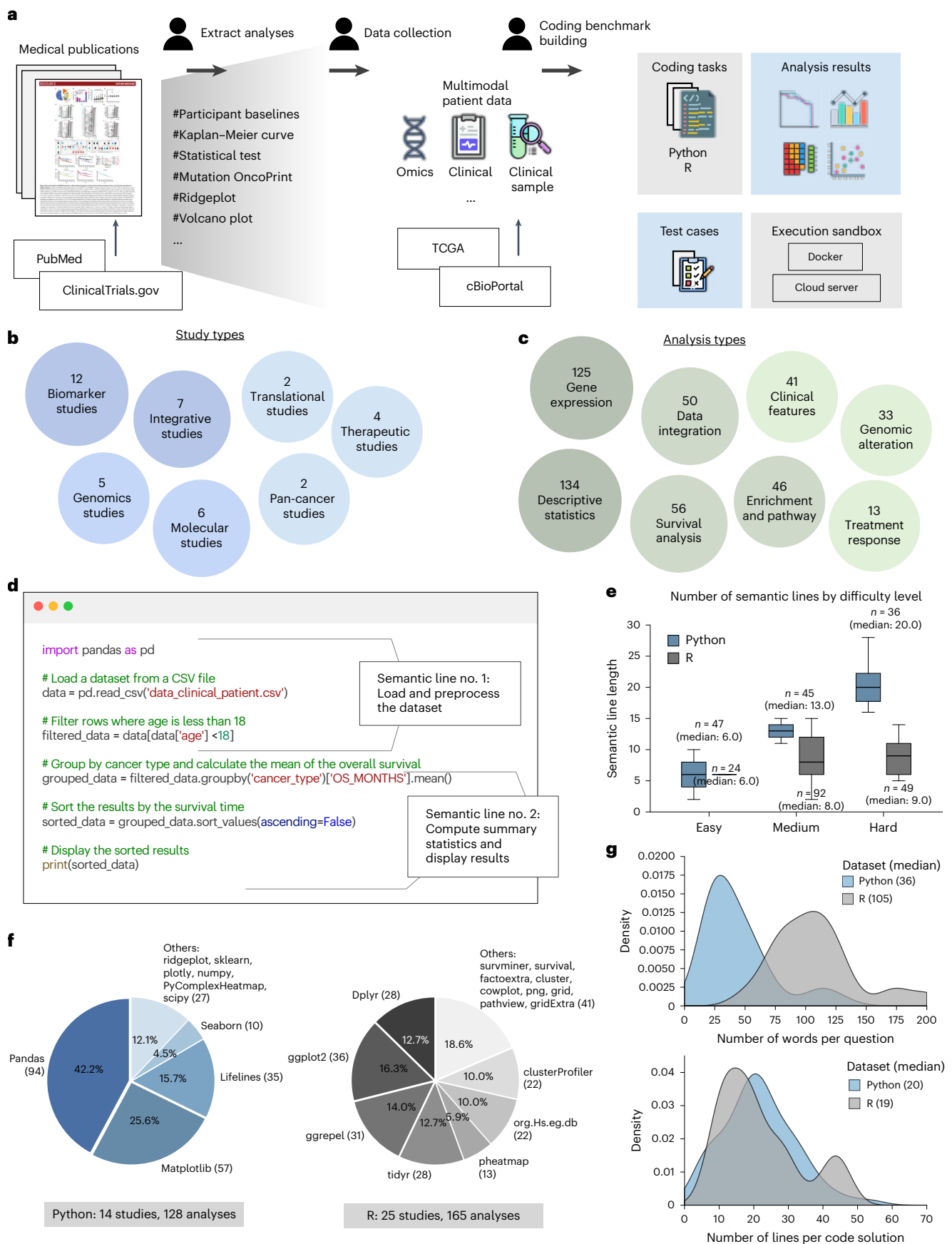
LLMs are not yet ready for fully automated data science

For benchmarking, we selected eight cutting-edge proprietary LLMs: GPT-4o²⁴, GPT-4o-mini²⁵, Sonnet²⁶, Opus²⁷, Gemini-Pro²⁸, Gemini-Flash²⁸ and o3-mini²⁹. We have also selected open-source LLMs in the comparison: Llama-3-8B, Llama-3-70B³⁰, DeepSeek-R1³¹, CodeLlama-7B, CodeLlama-34B³², StarCoder2-15B⁷, Qwen-Coder-7B and Qwen-Coder-32B³³. To explore their effectiveness and potential room for improvement, we applied several adaptations: chain-of-thought³⁴, few-shot prompting³⁵, automatic prompting³⁶, self-reflection¹¹, RAG³⁷ and ReAct agents³⁸ (see implementation details in Methods). We evaluated LLM performance using the Pass@1 metric, which measures the proportion of generated code that passes all test cases on the first attempt (see Methods for a detailed definition).

Figure 2b,c presents the code accuracy on Python and R tasks, respectively. First, no single prompting strategy consistently outperforms others across models, underscoring the dominant influence of the underlying LLM’s capabilities. For example, GPT-4o consistently outperforms GPT-4o-mini across all difficulty levels and adaptation strategies. Second, earlier-generation open-source LLMs (before mid-2024) failed to complete any tasks, whereas the latest open-source models released after 2025 achieve performance on par

Fig. 1 | Framework overview. **a**, We collect medical publications that are released with associated research data. From these, we extract and categorize the performed analyses and retrieve the corresponding omics and clinical data. Analyses are stratified by type and publication source to construct a candidate list for benchmarking. Expert data scientists then develop reference code solutions, from which testing cases and benchmark questions are derived. TCGA, The Cancer Genome Atlas. **b**, The benchmark dataset spans diverse biomedical study types, each represented as a circle with the corresponding sample count. **c**, Analysis types used to sample coding questions in the benchmark. Each circle represents one analysis category and its frequency. **d**, We estimate coding difficulty by calculating the number of operations in a reference solution. **e**, The distribution of semantic lines in code across difficulty levels. Difficulty is

defined via semantic line count and language model assessment. Python code generally involves more operations. In the box plot, the central line indicates the median, and the box represents the interquartile range (IQR; 25th–75th percentile). **f**, Key software packages used in the reference code solutions. For Python tasks, commonly used packages include pandas for tabular data manipulation; matplotlib, seaborn and plotly for data visualization; lifelines for survival analysis; and sklearn and scipy for machine learning and scientific computing. For R tasks, frequently used packages include ggplot2 for plotting, survminer and survival for survival analysis and clusterProfiler and cluster for gene set enrichment and clustering analysis. This demonstrates the benchmark’s breadth and complexity. **g**, The distribution of question lengths and code lengths for Python and R tasks.



with proprietary models. For example, DeepSeek-R1, which features enhanced reasoning capabilities and 671B parameters, outperforms most proprietary models. Third, overall accuracy remains limited across all LLMs: for Python tasks, accuracy ranges from 0.40 to 0.80 on easy, 0.15 to 0.40 on medium and 0.05 to 0.15 on hard tasks; for R tasks, accuracy ranges from 0.38 to 0.83 on easy, 0.26 to 0.66 on medium and 0.10 to 0.31 on hard tasks. These results highlight substantial room for improvement across all model classes.

Figure 2f further breaks down performance across the eight analysis types. Overall accuracy ranges from 0.12 to 0.42, with genomic alteration profiling and survival outcome analysis emerging as the most challenging, while data integration and transformation are among the easiest. The results reinforce the importance of base model capabilities: reasoning-enhanced models, such as o3-mini and DeepSeek-R1, consistently rank among the top performers and often exceed the average by a notable margin. For example, in genomic profiling, o3-mini achieves 0.37 compared with the average of 0.12; in survival analysis, it reaches 0.44 versus an average of 0.21. Despite these relative gains, the absolute performance across all analysis types remains unsatisfactory. Similar trends are observed in Supplementary Note 3.1, where allowing LLMs up to five attempts per question leads to only modest gains in success rates.

We analysed and categorized the error logs, as shown in Fig. 2d,e, for Python and R tasks, respectively. A notable finding is that the most frequent error type is ‘test failure’: cases where the generated code executes without error but contains hidden logical flaws that lead to incorrect analysis results. This is particularly concerning in high-stakes biomedical applications, where less experienced users may falsely assume that executable code is also correct. These findings highlight the need for greater transparency in LLM-generated code, enabling users to inspect better and verify the validity of analysis outputs.

Unlocking the power of LLMs through strategic adaptations

We manually reviewed the failure modes from previous experiments. We found that a major portion resulted from variability in how LLMs interpret the input request and the structure of the underlying dataset. As in Fig. 2d,e, data operation failures are the second most frequent errors in general (see example error cases in Supplementary Note 4). For failures due to explicit bugs, we hypothesize that advanced adaptations, such as adding more detailed instructions or examples to the prompt, or enabling agent-based workflows with iterative self-reflection and exploration, may help to reduce errors and improve performance. For failures due to implicit logical flaws, we propose that a structured, analysis plan-driven approach is essential. As shown in Fig. 2a, the analysis plan defines the specific operations required to fulfil the request. When provided before code generation, such structured input can reduce ambiguity, improve reproducibility and offer a more transparent path for inspecting and validating analysis outputs.

To make a more comprehensive comparison of adaptations, we further evaluated few-shot prompting (Few-shot), RAG and analysis plan as input prompt (PlanPrompt). RAG enables the LLM to query external sources (for example, Google search) to retrieve relevant

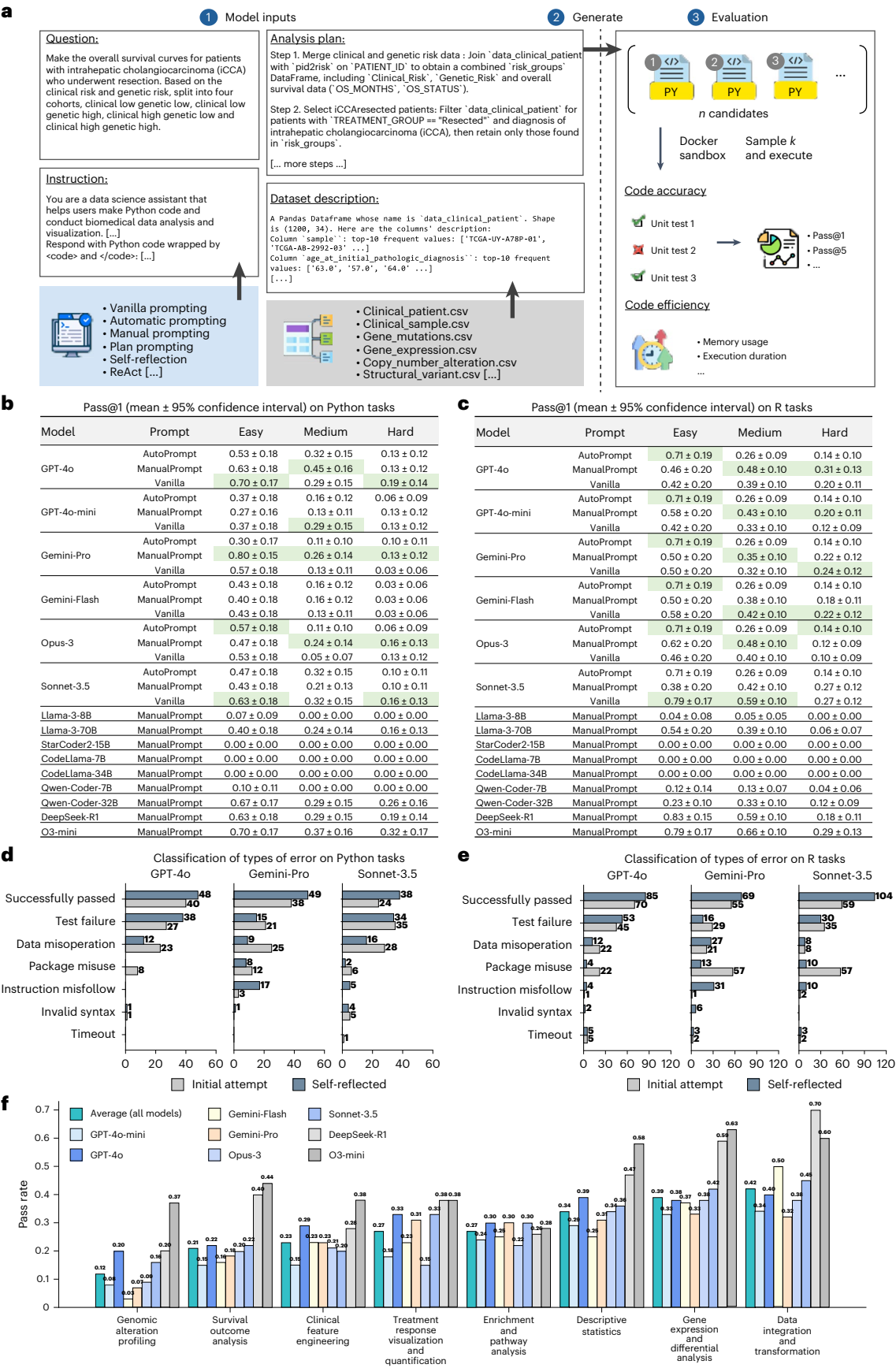
content such as package documentation, StackOverflow discussions and clinical background knowledge for one round before code generation. Few-shot prompting provides the model with several example question–answer pairs from the training set. PlanPrompt takes the analysis input as additional input asking LLMs to follow the plan to implement the code. Besides, we implemented three AI agents for comparison: the CoderAgent, self-reflection agent and DSWizard agent (Fig. 3a). The CoderAgent equips the LLM with a sandboxed execution environment, allowing it to iteratively generate, run and refine code while exploring the patient dataset. The self-reflection agent is the constrained CoderAgent that only updates the code answer in each round, taking the last-round execution outputs as input. The DSWizard agent extends this capability by incorporating a structured analysis plan as input. It is designed to iteratively refine the plan, validate the dataset schema against the plan and finally translate the completed plan into executable code. All are tested on the basis of a GPT-4o model. See Methods for their implementations; the prompts are listed in Supplementary Note 6.

Figure 3b compares the average accuracy of various adaptation strategies against Vanilla prompting across 11 held-out studies, with Fig. 3g further breaking down performance by task difficulty. Overall, none of the prompting adaptations demonstrates notable improvements over the Vanilla baseline. Both AutoPrompt and Few-shot prompting show limited generalization from the small training set to new analysis questions. The RAG approach also offers minimal gains. Upon examining the retrieved content, we found that it often lacked the relevance or specificity needed for the target analysis questions, underscoring the need to refine both the knowledge sources and the retrieval process to make RAG more effective for this task. Manual-Prompt provides a modest benefit, increasing Pass1 by an average of 10%. However, the overall impact remains limited because the predefined instructions cannot cover all edge cases and LLMs can still fail to follow them occasionally. By contrast, PlanPrompt shows a notable improvement, outperforming the best prompting method, Manual-Prompt, by 0.13 and 0.35 on medium and hard tasks, respectively. This underscores that the cornerstone of successful AI-assisted analysis lies in constructing a clear, actionable analysis plan.

Agents are not a panacea for this task. As shown in Fig. 3e,g, a CoderAgent based on ReAct often performs worse than single-round Vanilla prompting. In theory, this agent has access to full dataset schemas and can iteratively refine its code using a sandbox environment (Fig. 3a). In practice, this flexibility leads to inefficiencies: either spending excessive steps on unproductive data exploration or prematurely committing to incorrect final answers. By contrast, we explored a more constrained agent that performs self-reflection solely on its initial code attempt. We evaluated its performance across one to five reflection rounds, tracking Pass@1 improvements over time. As shown in Fig. 3c,d, this method clearly outperforms the CoderAgent. Furthermore, Fig. 2d,e reveals that self-reflection is more effective in addressing explicit bugs, such as package misuse and data operation errors, although it remains less effective for resolving implicit logical flaws that result in test failures.

Fig. 2 | Assessment of different models and adaptation methods in automating biomedical data science tasks. **a**, An overview of the input components provided to LLMs and the evaluation process. Question: defines the analysis objective and specifies the expected output format. Instruction: the prompt that guides LLM behaviour, varied across prompting strategies. Analysis plan: a user-defined, step-by-step plan that LLMs can follow and refine. Dataset description: textual metadata describing patient tables, essential for generating schema-aligned code. **b,c**, Pass@1 scores of eight proprietary LLMs and eight open-source LLMs across difficulty levels for Python (**b**) and R (**c**) coding tasks. Green shading highlights the best-performing prompting method at each task level, and bold values indicate the top-performing model for each difficulty level. Values after $\pm$ represent 95% confidence intervals. **d,e**, The

distribution of failure modes before and after self-reflection in Python (**d**) and R (**e**) tasks. Error types include: test failure (code runs but fails for testing cases), data misoperation (code cannot execute due to inappropriate operations on the data tables), package misuse (code contains an illegitimate usage of an external package), instruction misfollow (the LLM fails to follow the instruction hence refuses to generate the answer or generates wrong format code), invalid syntax and timeout (execution exceeds 3 min). **f**, Task-level Pass@1 performance of representative LLMs across eight biomedical data science task categories, showing that reasoning-oriented models consistently achieve the highest accuracy across diverse analytical tasks, particularly for complex workflows such as survival outcome analysis, gene expression analysis and data integration and transformation.



The findings above point to two keys: (1) a more constrained and purpose-optimized agent can outperform a highly flexible counterpart, and (2) the analysis plan is essential for guiding LLMs effectively. These insights are embodied in the DSWizard agent, illustrated in Fig. 3a. Unlike the CoderAgent, DSWizard focuses on iteratively refining the input analysis plan and translating it into executable code at the final step. It also integrates dataset schema access through RAG over a structured schema documentation index. As shown in Fig. 3b, DSWizard achieves a 100% win rate over Vanilla prompting, with an average accuracy of 0.74. Furthermore, Fig. 3g shows that it outperforms the PlanPrompt method by 0.27, 0.18 and 0.07 across easy, medium and hard tasks, respectively.

Human–AI collaboration boosts productivity for data science in biomedical research

Motivated by our experimental findings, we developed a platform to integrate LLMs into the biomedical data science workflow, as illustrated in Fig. 4. The platform provides an integrated interface that enables interactions with AI agents to (1) iteratively construct and refine an analysis plan and (2) convert the plan into executable code and iteratively improve it. The platform features a built-in sandbox environment that enables immediate code execution and result visualization. This design removes the need for users to manually craft complex prompts or switch between chat interfaces and external coding environments. By following an analysis plan-driven approach, the platform also enhances transparency, allowing domain experts to understand, inspect and iteratively refine the analysis process.

We involved five medical researchers in our user study. Each participant was assigned three studies^{39–41}, with approximately ten data analysis questions per study (Fig. 5d). Users worked with a GPT-4o-based agent on our platform to complete these tasks and submitted their solutions once their code passed all the test cases. The difficulty levels of the tasks were quantified, with the distribution shown in Fig. 5b. During the study, we tracked two core actions: code generation and code improvement (debugging) requests. The statistics of these user behaviours are depicted in Fig. 5b, where most users completed the first two studies, and a few tackled the third. After the study, we analysed the logs to compare the LLM-generated code with the final code solutions submitted by the users (Fig. 5a). In addition, we conducted a survey to gather their feedback on the platform and their experience. The survey questions were built on the basis of the Health Information Technology Usability Evaluation Scale (Health-ITUES)⁴².

The results of the code comparison analysis are presented in Fig. 5c, showing the distribution of the proportion of user-submitted code derived from LLM-generated solutions. We found that a considerable portion of the user-submitted code was drawn from LLM-generated code. For easy tasks, the median proportions were 0.88, 0.87 and 0.84 across the three studies, indicating that users heavily relied on LLM-provided solutions when crafting their final submissions. For medium and hard tasks, the ratios were generally lower: in study 1, the

proportions were 0.44 for medium tasks and 0.96 for hard tasks, while in study 2, the proportions were 0.75 for medium and 0.28 for hard tasks. These findings demonstrate the platform's potential to streamline the data science process for domain users with limited programming expertise. The quantitative survey results in Supplementary Note 5 also show an overall positive feedback from the involved participants.

Discussion

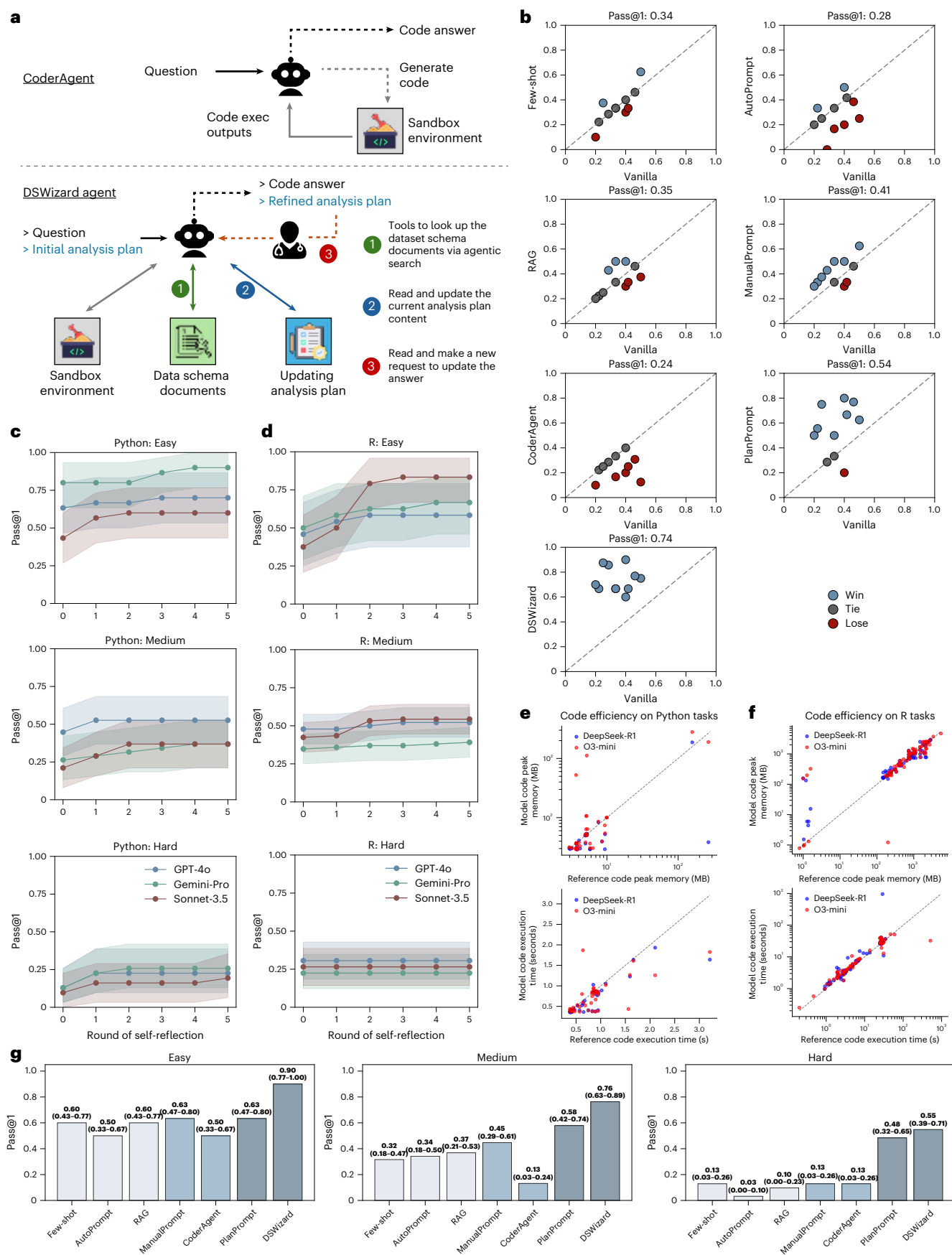
LLMs have emerged as powerful generalist AI capable of following human instructions to perform a wide range of tasks in medicine^{43–45}. In parallel, LLMs have demonstrated strong capabilities in solving coding challenges⁴, completing software engineering tasks⁴⁶, performing basic data analysis¹⁷ and validating research hypotheses^{20,21}. These advancements have led researchers to increasingly adopt LLMs and AI agents in their daily work. While this promise is real, it also carries considerable risks: AI-generated analyses can contain subtle errors masked by excessive outputs and visualizations, posing a serious threat of propagating incorrect scientific findings.

The first aim of this study is to rigorously benchmark how LLMs perform on real-world biomedical data science tasks. We build BioDS-Bench based on 39 published medical and biomedical studies, which has 293 data analysis tasks in both Python and R. Our experiments show that the current LLMs usually yield suboptimal performance, as they solved only 40–80% of easy tasks, 15–40% of medium tasks and 5–15% of hard tasks. Diving into the success and failure cases, we identified a key to successful data analysis with AI: building a structured analysis plan before coding. Experiment shows such a planning-augmented agent is able to achieve 90% on easy tasks, 76% on medium tasks and 55% on hard tasks, yielding absolute improvements over the best non-planning baselines by 27%, 31% and 42%, respectively. Nonetheless, a substantial gap remains before achieving fully automated biomedical research. BioDSBench provides a testbed for driving future development and evaluation.

The second aim is to advance towards more controllable and transparent AI assistants for biomedical data science. While general AI programming tools such as Cursor support software development effectively, biomedical data science poses distinct challenges. Our experiments show that, beyond code generation, making AI useful for biomedical data science requires handling variability in the request, structured reasoning over data semantics, deliberate analysis planning and domain-specific statistical workflows. This echoes the vision proposed by a recent position paper⁴⁷. Informed by these insights, we developed a platform that adopts a plan-driven approach to human–AI collaboration. It supports iterative planning, validates schemas and integrates code execution and visualization in a sandbox environment. This design lowers barriers for domain experts and promotes transparency and reproducibility. We list example cases in Supplementary Note 7 to show how an analysis plan brings better transparency and improves the LLMs' output quality. In a user study with five medical researchers, participants completed substantial portions of analyses via natural

Fig. 3 | Exploration of strategic adaptations and their effectiveness. **a**, The illustrative comparison between the CoderAgent and the proposed DSWizard agent for biomedical data science tasks. The CoderAgent follows a ReAct-style architecture, where an LLM is equipped with a sandbox environment to iteratively generate, execute and refine code based on dataset exploration. By contrast, the DSWizard Agent introduces three key enhancements: (1) a data schema index tool that allows direct retrieval of dataset schema descriptions without code execution; (2) a structured analysis plan input that clearly defines key steps and deliverables; and (3) plan updating during exploration, culminating in a refined natural language plan that is then converted into final code answers. **b**, Study-level code accuracy across adaptation methods. Each circle represents one study (with approximately ten questions). The x axis shows performance using Vanilla prompting, while the y axis shows the performance using the adaptation method. The diagonal line indicates parity. Results show

that providing a structured analysis plan substantially improves code quality: PlanPrompt achieves Pass@1 of 0.57, while DSWizard reaches 0.74, with 100% of the studies outperforming Vanilla prompting. **c,d**, Pass@1 performance with increasing rounds of self-reflection for Python (**c**) and R (**d**) tasks. In each round, the agent uses testing logs, runtime outputs and print statements to iteratively refine the code. Data are presented as mean values with 95% confidence intervals shown in the error bands. **e,f**, Pairwise comparison of memory usage and execution time between human-written reference code and LLM-generated code in Python (**e**) and R (**f**) tasks. No substantial differences are observed in terms of space and time complexity. **g**, Accuracy of different adaptation methods stratified by task difficulty. DSWizard achieves 90% accuracy on easy tasks, 76% on medium tasks and 55% on hard tasks, yielding absolute improvements over the best non-plan baseline by 27%, 31% and 42%, respectively. The values in parentheses indicate the 95% confidence intervals.



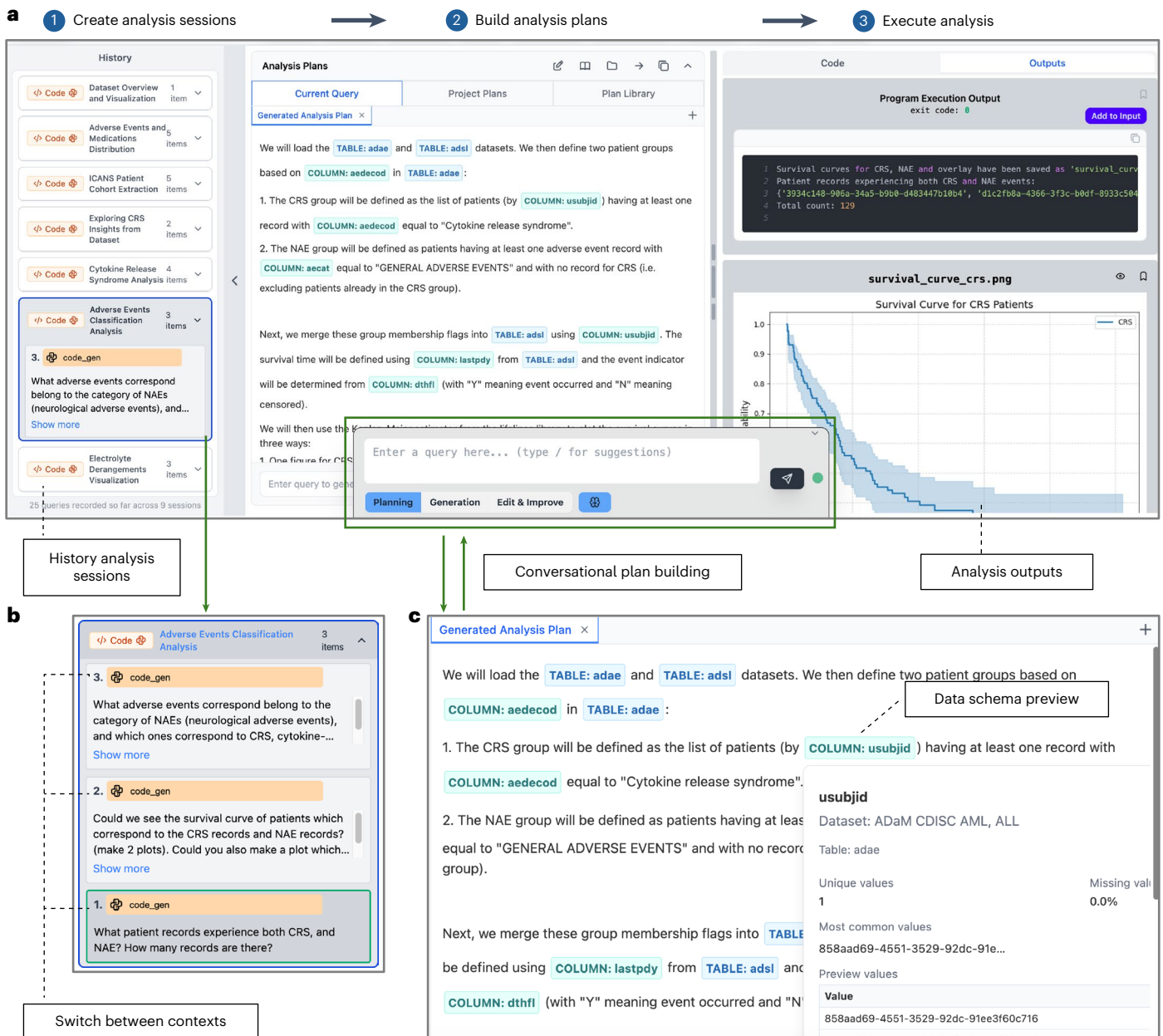


Fig. 4 | Overview of the developed biomedical data science platform. a, The developed platform guides users through three key steps: (1) Create analysis sessions, where each session is centred on a target analysis task. Within a session, all historical states are recorded, and users can navigate between them to revisit or resume prior work. (2) Build analysis plans, where users collaborate with the agent to explore the dataset and iteratively construct a structured analysis plan. (3) Execute analysis, where users convert the plan into executable code, run

the code to inspect output figures and further refine the plan until the target is achieved. **b**, The expanded session tab displays a list of historical project states, allowing users to conveniently return to previous checkpoints. **c**, The analysis plan section presents the generated plan in natural language. Users can manually edit the plan or request automatic refinement from the agent. In addition, the section supports checking the referenced columns and tables to validate the plan content.

language interactions, highlighting the value of domain-specialized systems. We also summarized practical usage guidance for integrating LLMs into data science workflows, offering design insights useful for AI developers building such platforms, and for biomedical researchers aiming to use AI assistants more effectively.

This study has several limitations. First, to ensure the quality of the benchmark, we manually created all the questions and solutions for the analysis tasks, which restricted our ability to scale the benchmark to cover more studies. A larger dataset with more coding tasks would enhance evaluation and be used for training LLMs specifically for data science tasks. Second, the user study results may be biased, as the participants were primarily medical researchers who,

while knowledgeable in their domain, had varying levels of coding proficiency. The usage patterns might differ if data scientists were the users, as they possess more advanced coding skills but know less about medicine. Third, the patient-level data in our testing set are publicly available, but privacy risks must be carefully considered when deploying LLMs for real-world biomedical data analysis. A recommended approach would be to separate the environment running code on sensitive patient data from the environment where LLMs are used. LLMs should access only the dataset schema or global statistics, without access to individual patient data. Finally, the benchmark primarily comprises analyses and patient data from oncology studies. While oncology is a critical domain in biomedical research, future

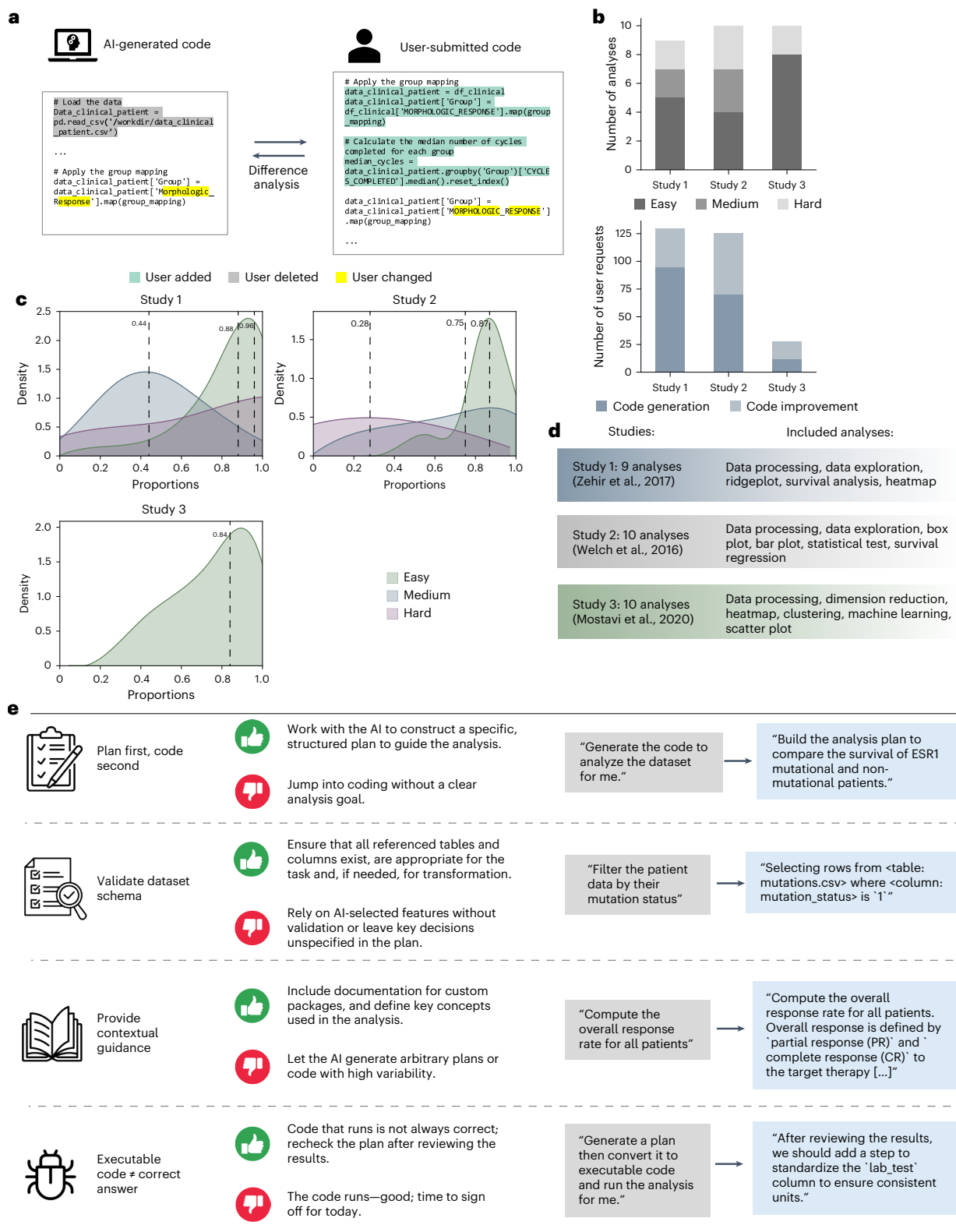


Fig. 5 | Overview of the user study. a, We compare the LLM-generated code captured in our logging system with the user-submitted answers to highlight the modifications made by users. This code difference analysis estimates the additional effort required from users to make the code functional. **b**, Statistics on task difficulty levels and user operations on the platform. Tasks span all three difficulty levels. ‘Code generation’ refers to user requests to generate code from a plan, while ‘code improvement’ refers to debugging or rewriting the code after execution. **c**, The distribution of the proportion of user-submitted code that was copied directly from LLM-generated code. The dotted vertical

lines show the medium of the proportions for each curve. **d**, The target studies completed by users, comprising 29 analysis tasks across diverse types such as ridge plots, survival analysis, statistical testing, clustering and machine learning. **e**, Actionable insights derived from analysing failure patterns in code generation and the user study. Four key categories emerged: the need for structured planning, validating data schema, incorporating contextual guidance and checking logical consistency in the plan. Each insight is illustrated with a problematic user request (in a grey box) and a recommended improved version (in a green box).

expansions could include broader data types such as single-cell and spatial transcriptomics, as well as medical imaging.

Methods

Selecting studies for building the benchmark

We created the testing dataset, referred to as BioDSBench, based on published medical studies and their associated patient-level datasets from TCGA⁴⁸ and cBioPortal²³. cBioPortal is a comprehensive database for cancer and genomics research, providing access to hundreds of studies with linked patient data. These datasets encompass various modalities, including clinical data, clinical sample data, mutations, copy number alterations, structural variants, RNA, mRNA and tumour miRNA, among others. This set-up ensures that the coding tasks in BioDSBench are closely aligned with the real-world challenges faced in biomedical data science, using authentic data and analysis tasks.

In constructing BioDSBench, we aimed to ensure broad coverage across major types of biomedical studies that reflect diverse research objectives and methodologies. To balance comprehensiveness with relevance, we selected source studies based on their alignment with the following well-established categories:

- **Genomics:** studies focused on large-scale genomic profiling of tumours using high-throughput sequencing to catalogue somatic mutations, copy number variations and related alterations. Example: PubMed ID 28472509.
- **Biomarkers:** research dedicated to discovering and validating diagnostic and prognostic biomarkers. These studies aim to support early detection, predict disease outcomes and guide treatment decisions. Example: PubMed ID 37350195.
- **Therapeutics:** publications investigating the relationship between genomic alterations and therapeutic responses, often identifying biomarkers or mechanisms for precision oncology. Example: PubMed ID 32864625.
- **Integrative:** studies combining multiple omics layers: genomic, transcriptomic and proteomic, to derive holistic insights into tumour biology. Example: PubMed ID 29713087.
- **Translational:** research applying molecular findings to clinical contexts, aiming to improve diagnostics, prognostics and treatment decisions. Example: PubMed ID 30867592.
- **Molecular:** work examining transcriptomic, epigenomic and proteomic features in both patient samples and cancer cell lines beyond DNA mutation analysis. Example: PubMed ID 27959731.
- **Pan-cancer:** studies performing cross-cancer analyses to identify shared and distinct molecular patterns across tumour types. Example: PubMed ID 28481359.

With the defined study taxonomy, we selected 39 studies where each study belongs to one of the above study types. A list of example selected studies can be found in Supplementary Fig. SN1.1.

Extracting and categorizing analyses from the selected studies

Based on the selected studies, our objective is to create data science questions that closely mirror the actual analyses performed by the original authors, capturing the common practices across biomedical research. To this end, we developed a taxonomy by systematically extracting and summarizing the key data analyses presented in the main figures and tables of these studies. This taxonomy encompasses eight analysis categories: (1) gene expression and differential analysis; (2) genomic alteration profiling; (3) survival outcome analysis; (4) clinical feature engineering; (5) treatment response visualization and quantification; (6) descriptive statistics; (7) enrichment and pathway analysis; and (8) data integration and transformation. Below, we detail each category and illustrate them with representative examples from the studies.

Gene expression. This category includes tasks involving the processing, comparison and visualization of gene expression data. Typical analyses include merging expression matrices with clinical labels, filtering differentially expressed genes (DEGs) based on $\log_2$ fold change and statistical significance, and visualizing expression profiles using heatmaps, principal component analysis or boxplots. For example, one task filters DEGs with $\log_2$ fold change >1 and P value <0.05 , while another plots principal component analysis results coloured by cancer type.

Genomic alteration. This category focuses on characterizing somatic alterations such as mutations and copy number alterations. Analyses often include classifying mutation subtypes (for example, missense, truncating and in-frame), computing mutation burdens or substitution frequencies (for example, $A > G$ and $CC > TT$) and annotating copy number events (for example, amplification and deletion). Representative tasks include generating an alteration matrix for truncating and missense mutations, or plotting oncoprints of frequently altered genes across patients.

Survival analysis. Tasks in this category investigate the association between molecular or clinical features and patient survival outcomes such as overall survival (OS), progression-free survival (PFS) or time to treatment failure. Analyses include Kaplan–Meier curve estimation, log-rank tests and Cox proportional hazards modelling. For instance, one analysis generates OS survival curves stratified by unsupervised clusters, and another computes P values for OS and PFS hazard ratios across gene-defined subgroups.

Clinical features. This category includes the derivation of structured features from raw clinical or biomarker data. It encompasses scoring systems such as the International Prognostic Index, PD-L1 and HER2 status classifications, and tumour mutational burden computation. Tasks may involve converting raw scores or measurements into categorical indicators. For example, one analysis classifies patients into International Prognostic Index risk groups, and another calculates average tumour mutational burden from mutation data.

Treatment response. This class captures tasks focused on summarizing and visualizing patient-level treatment responses. Typical analyses include computing response rates (for example, Response Evaluation Criteria in Solid Tumors (RECIST) and positron emission tomography (PET)-defined) and generating waterfall plots (for example, maximum tumour shrinkage) or swimmer plots (for example, treatment duration annotated with off-study reasons). One task computes the overall response rate based on RECIST categories, while another creates a waterfall plot showing the percentage change in tumour size per patient.

Descriptive statistics. This category includes statistical summaries of patient or sample characteristics, often stratified by groups. It covers simple counts, frequencies, distributions and group comparisons using statistical tests such as t -tests or Wilcoxon rank-sum tests. For instance, one analysis counts how many patients achieved complete or partial remission, while another compares median diagnosis age between early-onset and late-onset cohorts.

Enrichment and pathway. This category includes tasks that interpret gene sets in the context of biological functions using enrichment methods. Analyses typically involve performing Gene Ontology or Kyoto Encyclopedia of Genes and Genomes enrichment using tools such as `enrichGO()`, mapping gene symbols to identifiers (for example, ENTREZ IDs) and visualizing results with dot plots or bar charts. These tasks provide functional insights into differentially expressed or mutated genes. For example, one task performs enrichment for DEGs and visualizes the Gene Ontology terms with colour-coded P values and size-scaled GeneRatios.

Data integration. This category involves preprocessing operations that prepare data for analysis, such as merging tables, transposing matrices or generating indicator variables. These tasks often serve as building blocks for more complex analyses. Representative tasks include joining gene expression data with cluster assignments, transposing wide-format tables or constructing binary matrices indicating mutation presence across patients and genes.

Creating the questions and answers

Each extracted analysis is assigned to at least one analysis type in our taxonomy. We stratified the analyses by both analysis type and study, selecting a representative subset of approximately 300 analyses, with each study contributing five to ten analyses. For each selected analysis, we manually constructed coding tasks that reflect the sequence of data processing steps used to generate the findings in the original study. Although the source studies may be included in the LLMs' training data, the specific coding tasks and reference solutions were newly created for this benchmark and are not directly extractable from the original texts, thereby minimizing the risk of data leakage. The tasks are organized into five key components: the input question, a description of the patient dataset schema, prefix code, reference solution and test cases. An example of a coding task is shown in Supplementary Fig. SN2.1.

To enable automatic testing, it is essential to maintain strict consistency between the input question and the corresponding test cases, particularly in terms of output naming and formatting. For example, a general prompt such as 'Describe the gene mutations in the dataset' is inherently open-ended and can yield a wide range of responses. A common approach might be to compute mutation counts in all patients; however, without a clearly defined analytical objective and expected output format, such variability hinders a reliable evaluation because an answer different from that is not necessarily a wrong answer. To address this, each question in BioDSBench is explicitly structured into two components: (1) a task description that specifies the analysis to be performed, and (2) an output format requirement that constrains the expected answer. For instance, a refined version of the above question would state: 'Describe the prevalence of the TP53 gene mutation across different cancer subtypes' with the output requirement: 'Return the results as a `pd.Series` named `output_df`, sorted by mutation count in descending order.' This clear specification allows us to construct deterministic test cases that validate the output of LLM-generated code against a standardized solution.

The prefix code refers to the prerequisite code necessary to run before addressing a specific question. This approach mirrors the workflow of data scientists working in computational notebook environments such as Jupyter Notebook⁴⁹, where certain data processing steps are required for multiple analyses. However, it would be redundant and inefficient to repeat these steps for every coding task. For example, in one step, a data scientist might merge the patient clinical data table with the mutation table to link patient outcomes with gene mutation information. This merged dataset is then used in subsequent analyses, such as survival analysis grouped by gene mutations. For these follow-up tasks, the LLMs are not required to repeat the data merging process. Instead, the merging code is provided as prefix code, allowing the LLMs to build on the processed data and focus on the specific task at hand. This structure ensures efficiency and mimics how data scientists typically manage code dependencies across related tasks.

To protect the privacy of patient records, it is crucial to handle how patient data is passed in prompts to proprietary LLMs, such as OpenAI's GPT models, for coding tasks. Our approach avoids using individual-level patient records as input. Instead, we use a template to generate a caption for each dataset. This caption includes the table's name, its dimensions (shape), the names of all columns, and representative values from each column. This method ensures that no private or sensitive information about individuals is shared, while still enabling LLMs to understand the dataset's structure and content sufficiently to synthesize code for data science.

Testing cases were developed on the basis of the execution results of the manually created reference code answers. We developed specific testing cases for various types of output required to answer the input data science questions, including numerical, categorical, dataframes and object outputs. For numerical outputs, such as the number of patients (integers) or average age (continuous values), the testing cases check for exact matches in integers and verify that the absolute difference for continuous values falls within an acceptable error range. For categorical outputs, such as a list of the top ten frequently mutated genes, the testing cases ensure that the generated list matches the expected set. For dataframe outputs, such as merging two tables, we clarify the expected column names and content in the question. The testing cases then verify the shape of the resulting dataframe and check global statistics for each column, depending on the variable types. When the expected outputs are special objects, such as in visualization tasks, where the output is a figure, we specify in the question which tools should be used to create the visualization. The testing cases then check if the specified tools were used correctly. In addition, we include instructions to save the inputs used for plotting, allowing us to verify the correctness of these inputs as a proxy for validating the accuracy of the visualizations.

Estimating and classifying question difficulties

We measure the difficulty of each question in BioDSBench by considering the degree of efforts and the levels of expertise required to implement the code answers. Specifically, to estimate the efforts, we measure the number of semantic operations, that is, distinct reasoning or coding steps, required to arrive at the final answer. For example, a simple question such as 'Load the gene mutational data' maps to a single operation (for example, reading a table into memory). By contrast, a more complex question like 'Plot survival curves for PFS and OS in 1p19q codeleted patients' requires multiple reasoning steps: (1) load clinical and sample metadata, (2) filter for 1p19q codeletion, (3) preprocess survival status columns, (4) generate OS curves and (5) generate PFS curves. While each step may consist of multiple lines of code, we treat them as one semantic operation if they collectively serve a single analytical purpose.

We used GPT-4o to parse expert-written reference code and estimate the number of analytical operations involved. While LLMs have demonstrated promise in structural code understanding, we recognize the need to validate GPT-4o's reliability in this context. To this end, we manually audited the decomposed operations generated by GPT-4o for 118 Python questions. In all cases, the extracted operations were semantically consistent with the intended analytical steps, supporting GPT-4o's utility as a practical proxy for estimating task complexity. In addition, we assessed task difficulty using an LLM-as-judge approach, where the model directly classified each code example as easy, medium or hard. The prompts used for Python and R task difficulty classification are provided in Supplementary Notes 6.7 and 6.8, respectively.

LLMs and adaptation methods

We investigated a diverse range of LLMs for data science code generation tasks, focusing on cutting-edge proprietary models. These include proprietary models such as OpenAI's GPT-4o²⁴ and GPT-4o-mini²⁵, Google's Gemini-Pro and Gemini-Flash²⁸, as well as Anthropic's Opus-3²⁷ and Sonnet-3.5²⁶. Each of these models is a flagship proprietary LLM known for its strong performance in medical and biomedical tasks. In addition, all these models feature long context windows, enabling them to handle large inputs efficiently: GPT-4o and GPT-4o-mini support up to 128K tokens, Gemini-Pro supports up to 2M tokens, Gemini-Flash supports up to 1M tokens and both Opus-3 and Sonnet-3.5 support up to 200K tokens. This extended context capacity is essential for processing complex datasets and tasks typical in biomedical data science.

We have also expanded our comparison to include recent open-source LLMs to reflect the progress of the research community.

Specifically, we evaluated general-purpose models such as Llama-3-8B (Llama-3.1-8B-Instruct), Llama-3-70B (Llama-3.3-70B-Instruct)³⁰ and DeepSeek-R1³¹, as well as code-specialized models including StarCoder2-15B⁷, CodeLlama-7B (CodeLlama-7B-Instruct-hf), CodeLlama-34B (CodeLlama-34B-Instruct-hf)³², Qwen-Coder-7B (Qwen2.5-Coder-7B-Instruct) and Qwen-Coder-32B (Qwen2.5-Coder-23B-Instruct)³³. These models span multiple release periods, CodeLlama in mid 2023, StarCoder2 in early 2024, Llama-3 in late 2024, and Qwen-2.5-Coder and DeepSeek-R1 in early 2025, allowing us to examine both the temporal evolution of open models and their performance gap relative to proprietary LLMs in this domain.

In this study, we explored adaptation methods to guide pretrained generalist LLMs for specific tasks without fine-tuning the models. The primary reason for this approach was the limited dataset scale, which was sufficient only for testing purposes. In addition, including publicly available code examples from sources like GitHub would probably offer minimal benefit, as these LLMs have already been extensively trained on such data.

In-context learning. LLMs exhibit a remarkable ability to comprehend input requests and follow provided instructions during code generation. A key concept in this process is in-context learning (ICL), which allows LLMs to learn from examples and task instructions provided within the input context at inference time³⁵. ICL has become a major technique for adapting LLMs to medical tasks^{44,45,50}. In this study, we implemented ICL across all methods, as each input question contains specific instructions for the expected output format, which the LLMs use to generate responses that aim to pass testing cases. The prompts can be found in the ‘Implementations of prompts’ section in Supplementary Information. The Vanilla method represents the minimum prompt engineering to ask LLMs to answer the input coding question. We further enhanced this by incorporating additional expert knowledge into the prompts, which we refer to as the ManualPrompt variant, as shown in Figs. 2 and 3. In addition, few-shot prompting, a form of ICL, was used to guide LLMs to produce both high-quality and correctly formatted outputs. This was achieved by adding demonstrations of example input questions and output code solutions into the prompt, following the five-shot prompting technique. Consistent with prior findings⁵¹, we observed that using relevant examples is more effective than random ones. To optimize this, we dynamically retrieved examples most relevant to the input question by computing semantic similarity using OpenAI’s embedding model⁵². This ensured that the examples provided in the prompt closely aligned with the task at hand, improving LLM performance. This approach was identified as the few-shot variant in experiments shown in Fig. 3b.

Automatic prompting. LLMs were found able to suggest improvements for a given prompt with a given set of demonstration example inputs and outputs⁵³. In the experiments, the automatic prompt generation is performed using DSPy’s Optimizer framework for instruction refinement³⁶. The process consists of two main components: a prompt generator and an output evaluator. The prompt generator proposes multiple candidate prompts in each iteration. These prompts are used to guide the LLM in producing code solutions for benchmark tasks. The output evaluator assesses the quality of the generated code using two criteria: (1) a sandboxed execution environment to check whether the code runs successfully, and (2) an LLM-based judgement to evaluate the semantic correctness of the output. The final reward signal is a weighted combination of these two factors. DSPy uses a black-box optimization strategy to guide prompt search. It encourages exploration by generating diverse candidates and selects top-performing prompts based on the feedback from the evaluator. This automated process improves the quality and robustness of the generated instructions. The resulting prompts are referred to as the AutoPrompt variant in Figs. 2 and 3.

Retrieval-augmented generation. LLMs that rely solely on their internal knowledge often produce erroneous outputs, particularly due to outdated information or hallucinations. RAG addresses this issue by dynamically incorporating external knowledge into prompts during generation³⁷. In our experiments, we implemented RAG through an external application programming interface (API) that connects to the Google search engine via Vertex AI Search³⁴. We restricted our searches to medical-related sources, such as PubMed, and coding-related platforms, such as GitHub and StackOverflow. The top ten most relevant search results were retrieved and incorporated into the LLM’s prompt to assist with solving coding tasks. This forms the foundation of the RAG variant shown in the experiment results in Fig. 3.

Self-reflection. LLMs can produce flawed outputs on their first attempt, but they can improve through iterative feedback and refinement⁵⁵. This approach mirrors the natural process humans follow when programming, testing and debugging code⁴¹. In our experiments, we implemented self-reflection, allowing LLMs to attempt debugging their incorrect code solutions, with results shown in Fig. 3c. The process involved executing the initially generated code along with the testing cases and collecting output logs reflecting (1) errors from failed tests, (2) runtime errors within the code and (3) the printed values and shapes of intermediate variables. We then prompted the LLM to explain why the code was incorrect, propose a plan for correction and provide a revised code solution. This cycle was repeated up to five times, and in each round, only the unresolved questions were carried forward for further self-reflection. This iterative method allowed LLMs to gradually improve their code solutions over multiple attempts.

Plan prompting. We develop two methods that incorporate explicit analysis planning: PlanPrompt and Plan-ReAct. In the PlanPrompt method, the LLM is prompted to first generate a natural language analysis plan before proceeding to code generation. This plan outlines all necessary steps, including data inspection, transformation, modelling and evaluation, and the model is required to follow it strictly when generating the final code. PlanPrompt is a single-pass method: the plan is written once, reviewed by the user if desired and then used directly to guide code synthesis. This approach improves transparency by exposing the LLM’s reasoning in natural language, accuracy by allowing user correction of flawed logic before code is generated, and reproducibility by ensuring that future generations for the same query follow consistent logic and structure.

CoderAgent. The CoderAgent implements a classic ReAct agent³⁸ architecture tailored for data science tasks, consisting of a single LLM agent equipped with a code execution tool. In theory, this set-up allows the agent to access all necessary information about the dataset by generating Python code, executing it and interpreting the results. The agent can iteratively reason, act (by invoking the execution environment) and observe outputs such as printed values, data summaries or error traces. This closed-loop framework enables the agent to verify its assumptions, check code correctness and adjust its reasoning accordingly. Unlike plan-based methods, Basic-ReAct does not generate an explicit high-level plan in advance. Instead, it dynamically explores the solution space based on intermediate observations, allowing flexibility but also increasing the risk of drifting without a structured plan. This method serves as a baseline for assessing how well an LLM can self-navigate typical data analysis workflows when given access to an execution environment.

DSWizard agent. DSWizard agent builds upon both plan prompting and CoderAgent, combining structured high-level planning with iterative tool-assisted reasoning. In addition to requiring the LLM to first generate an explicit analysis plan, we equip the agent with an extended toolset beyond basic code execution. These tools include commands to

list available tables, inspect column names, retrieve dataset overviews and fetch detailed information about specific columns. This structured interface enables the agent to access and reason about the dataset in a more targeted and efficient manner.

We found these additional tools to be essential: while the CodeAgent theoretically has full access to data through code execution alone, in practice, they often struggle to generate correct exploratory code, especially in complex, multitabled biomedical datasets. As a result, they may require many inefficient exploration steps or fail to locate the relevant data entirely. By contrast, DSWizard benefits from both a global plan to guide the workflow and structured data access tools that reduce ambiguity and trial and error. This leads to more accurate, interpretable and efficient problem solving, particularly in data-rich and schema-heavy biomedical analysis settings.

Experimental set-up

All experiments were run in Python v3.10. The versions of key software are anthropic v.0.34.2, boto3 v.1.35.16, openai v.1.44.1, google-generativeai v.0.7.2, google-cloud-aiplatform v.1.65.0, dspai v.2.4.14, langchain v.0.2.16, vLLM v0.9.2, transformers v4.53.2 and docker v.7.1.0 with Python v.3.10. Specifically, we accessed OpenAI's models via OpenAI's platform, the Google models through Vertex AI provided in Google Cloud, and Anthropic's models through AWS's Bedrock APIs.

Sandbox development. We created a sandbox environment to enable the automatic execution and testing of code generated by LLMs. This was accomplished by creating a standardized Docker image that hosts both Python and R environments, allowing scripts in either language to be run via the command line. We utilized Pipenv to manage the Python environment, installing packages such as pandas and matplotlib. Similarly, for R, we defined necessary packages such as dplyr and survival to be installed when building the image. The sandbox interface dynamically builds Docker containers based on the defined image, accepts code strings from LLMs, converts them into Python or R scripts and then executes them. The sandbox also accepts dataset uploads, enabling parallel real-time code execution without impacting the main experimental environment. This set-up ensures a controlled and isolated environment for running and testing LLM-generated code safely and efficiently.

Platform development. For the user study, we developed a platform to facilitate human–AI collaborative coding for data science tasks, as illustrated in Fig. 4. The platform is designed to relieve users from setting up their coding environment and provide code suggestions based on natural language requests, while enabling real-time code execution and feedback. The primary window features a user input box where users can choose from various platform commands, which are categorized into two types: brainstorming and programming. In the brainstorming mode, users can interact with the LLM assistant to search medical publications from PubMed or perform general searches via Google. They can also collaborate with LLMs to develop plans for data analysis tasks. In the programming mode, users can either ask the LLM to generate code from scratch or request improvements or corrections to existing code. Users have the option to generate code in either R or Python. Once the code is generated, users can execute it within a sandbox environment. The platform provides execution logs and any produced artefacts, such as figures, directly to the frontend, allowing users to receive immediate feedback on the results. This process maximizes the utility of LLMs by enabling users to collaboratively plan data analyses and then guide LLMs to generate accurate code solutions. In the second window, users can select patient datasets and apply their generated data analysis code to gain insights. The platform allows users to preview tables, columns and values from the selected dataset, providing a streamlined experience for conducting data science tasks in a collaborative, AI-assisted environment.

Questionnaire design. We developed a questionnaire to gather feedback from users following the user study. The survey was designed on the basis of the Health Information Technology Usability Evaluation Scale (Health-ITUES)⁴², originally created to assess the usability of a web-based communication system for scheduling nursing staff. We adapted the questions in line with the spirit of the scale, covering four main topics: output quality, support and integration, system complexity and system usability. The original 22-item questionnaire was streamlined to 10 items, with users rating each on a 5-point Likert scale, ranging from strongly disagree to strongly agree. Moreover, we included an open-ended question, allowing users to provide free-text comments for further insights. This format ensured a concise yet comprehensive collection of user feedback on the platform's performance and usability.

Evaluation metrics and statistical analysis

The Pass@k metric was used to evaluate the performance of code generation in our study. Here, n represents the total number of code solutions generated, and c is the number of correct solutions, where $c \leq n$. Correct samples are those that pass all unit tests. The unbiased estimator⁴ for Pass@k is given by

$$\text{Pass@k} = \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]. \quad (1)$$

Pass@k ranges from 0 to 1 and estimates the probability that at least one of the k generated code samples for a given task passes all the unit tests. In our study, we used two metrics: Pass@1 and Pass@5. Pass@1 is a stricter metric, evaluating whether the LLM can solve the task on the first attempt. To ensure reproducibility, we set the LLMs' temperature to zero for this evaluation. For Pass@5, we allowed the LLMs to generate ten solutions for each question to estimate the likelihood of producing a correct answer within five attempts.

To compare the LLM-generated code with user-submitted or reference code solutions, we first parse the code string using abstract syntax trees to extract operators and variables, which allows for a structural analysis of the code. We then tokenize the code based on this parsing result. Using Python's difflib library, we compare the differences between two text sequences.

Let s_1 represent the tokenized LLM-generated code and s_2 represent the tokenized user-submitted code. We compute the length of overlapping tokens between the two sequences, denoted as s . The ratio of user-submitted code copied from LLM-generated code can then be calculated using the following formula:

$$\text{Copy ratio} = \frac{\text{length}(s)}{\text{length}(s_2)}. \quad (2)$$

This method quantifies the extent to which the user-submitted code overlaps with the LLM-generated code, providing insight into the level of influence the LLM had on the final solution.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

The curated data science tasks with the reference answers and testing cases in BioDSBench can be accessed at <https://huggingface.co/datasets/zifeng-ai/BioDSBench>. The anonymized patient data where these data analyses are performed are available via the cBioPortal website at <https://www.cbioportal.org/datasets> and the UCSC Xena website at

<https://xenabrowser.net/datapages/>. Source data are provided with this paper.

Code availability

Code for implementing and experimenting with the proposed methodology is available via GitHub at <https://github.com/RyanWangZf/BioDSBench>. The human–AI collaborative biomedical data science programming platform can be accessed via a web-based app²² and can be accessed per request at <https://keiji.ai/contact.html>. The demonstration video can be accessed at https://www.youtube.com/watch?v=c5ZJsFXQ_B0.

References

- Radenkovic, D., Keogh, S. B. & Maruthappu, M. Data science in modern evidence-based medicine. *J. R. Soc. Med.* **112**, 493–494 (2019).
- Ellis, L. D. To meet future needs, health care leaders must look at the data (science). *Harvard T.H. Chan School of Public Health* <https://www.hsph.harvard.edu/ecpe/to-meet-future-needs-health-care-leaders-must-look-at-the-data-science/> (accessed 16 September 2024).
- Meyer, M. A. Healthcare data scientist qualifications, skills, and job focus: a content analysis of job postings. *J. Am. Med. Inf. Assoc.* **26**, 383–391 (2019).
- Chen, M. et al. Evaluating large language models trained on code. Preprint at <https://arxiv.org/abs/2107.03374> (2021).
- Li, Y. et al. Competition-level code generation with alphacode. *Science* **378**, 1092–1097 (2022).
- Luo, Z. et al. Wizardcoder: empowering code large language models with evol-instruct. In *The Twelfth International Conference on Learning Representations* 1–21 (OpenReview, 2023).
- Lozhkov, A. et al. Starcoder 2 and the stack v2: the next generation. Preprint at <https://arxiv.org/abs/2402.19173> (2024).
- Zhang, F. et al. RepoCoder: repository-level code completion through iterative retrieval and generation. *The 2023 Conference on Empirical Methods in Natural Language Processing* 2471–2484 (Association for Computational Linguistics, 2023).
- Parvez, M. R., Ahmad, W., Chakraborty, S., Ray, B. & Chang, K.-W. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021* 2719–2734 (Association for Computational Linguistics, 2021).
- Wang, Z. Z. et al. CodeRAG-Bench: can retrieval augment code generation? In *Findings of the Association for Computational Linguistics: NAACL 2025* 3199–3214 (Association for Computational Linguistics, 2025).
- Chen, X., Lin, M., Schärli, N. & Zhou, D. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations* 1–80 (OpenReview, 2024).
- Austin, J. et al. Program synthesis with large language models. Preprint at <https://arxiv.org/abs/2108.07732> (2021).
- Hendrycks, D. et al. Measuring coding challenge competence with apps. In *Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track* 1–11 (OpenReview, 2021).
- Liu, J., Xia, C. S., Wang, Y. & Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. *Adv. Neural Inf. Process. Syst.* **36**, 21558–21575 (2023).
- Jimenez, C. E. et al. SWE-bench: can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations* 1–51 (OpenReview, 2024).
- Huang, J. et al. Execution-based evaluation for data science code generation models. In *Proc. Fourth Workshop on Data Science with Human-in-the-Loop (Language Advances)* 28–36 (Association for Computational Linguistics, 2022).
- Lai, Y. et al. DS-1000: a natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning* 18319–18345 (PMLR, 2023).
- Tayebi Arasteh, S. et al. Large language models streamline automated machine learning for clinical studies. *Nat. Commun.* **15**, 1603 (2024).
- Tang, X. et al. Biocoder: a benchmark for bioinformatics code generation with large language models. *Bioinformatics* **40**, i266–i276 (2024).
- Majumder, B. P. et al. DiscoveryBench: towards data-driven discovery with large language models. In *The Thirteenth International Conference on Learning Representations* 1–34 (OpenReview, 2025).
- Wang, Z., Danek, B. & Sun, J. BioDSA-1K: benchmarking data science agents for biomedical research. Preprint at <https://arxiv.org/abs/2505.16100> (2025).
- TrialMind Data Science Assistant. Keiji AI. <https://www.trialmindapis.com/api/data-science> (2025).
- cBioPortal for cancer genomics. *cBioPortal* <https://www.cbioportal.org/datasets> (accessed 17 September 2024).
- Hello GPT-4o. OpenAI <https://openai.com/index/hello-gpt-4o/> (accessed 17 September 2024).
- GPT-4o mini: advancing cost-efficient intelligence. OpenAI <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/> (accessed 17 September 2024).
- Claude 3.5 Sonnet. Anthropic <https://www.anthropic.com/news/claude-3-5-sonnet> (accessed 17 September 2024).
- Introducing the next generation of Claude. Anthropic <https://www.anthropic.com/news/claude-3-family> (accessed 17 September 2024).
- Reid, M. et al. Gemini 1.5: unlocking multimodal understanding across millions of tokens of context. Preprint at <https://arxiv.org/abs/2403.05530> (2024).
- OpenAI o3-mini: pushing the frontier of cost-effective reasoning. OpenAI <https://openai.com/index/openai-o3-mini/> (accessed 6 June 2025).
- Grattafiori, A. et al. The Llama 3 herd of models. Preprint at <https://arxiv.org/abs/2407.21783> (2024).
- Guo, D. et al. Deepseek-R1 incentivizes reasoning capability in LLMs via reinforcement learning. *Nature* **645**, 633–638 (2025).
- Rozière, B. et al. Code Llama: open foundation models for code. Preprint at <https://arxiv.org/abs/2308.12950> (2024).
- Hui, B. et al. Qwen2.5-coder technical report. Preprint at <https://arxiv.org/abs/2409.12186> (2024).
- Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **35**, 24824–24837 (2022).
- Brown, T. B. et al. Language models are few-shot learners. In *Proc. 34th International Conference on Neural Information Processing Systems*, 1877–1901. (Curran Associates, 2020).
- Khattab, O. et al. DSPy: compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations* 1–31 (OpenReview, 2024).
- Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020).
- Yao, S. et al. ReAct: synergizing reasoning and acting in language models. In *International Conference on Learning Representations* 1–33 (OpenReview, 2023).
- Zehir, A. et al. Mutational landscape of metastatic cancer revealed from prospective clinical sequencing of 10,000 patients. *Nat. Med.* **23**, 703–713 (2017).

40. Welch, J. S. et al. Tp53 and decitabine in acute myeloid leukemia and myelodysplastic syndromes. *N. Engl. J. Med.* **375**, 2023–2036 (2016).
41. Mostavi, M., Chiu, Y.-C., Huang, Y. & Chen, Y. Convolutional neural network models for cancer type prediction based on gene expression. *BMC Med. Genomics* **13**, 44 (2020).
42. Yen, P.-Y., Wantland, D. & Bakken, S. Development of a customizable health it usability evaluation scale. In *AMIA Annual Symposium Proceedings* Vol. 2010, 917 (American Medical Informatics Association, 2010).
43. Wang, Z. et al. Accelerating clinical evidence synthesis with large language models. *npj Digit. Med.* **8**, 509–523 (2025).
44. Lin, J., Xu, H., Wang, Z., Wang, S. & Sun, J. Panacea: a foundation model for clinical trial search, summarization, design, and recruitment. Preprint at <https://arxiv.org/abs/2407.11007> (2024).
45. Jin, Q. et al. Matching patients to clinical trials with large language models. *Nat. Commun.* **15**, 9074 (2023).
46. Wang, X. et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations* 1–8 (OpenReview, 2025).
47. Majumder, B. P. et al. Position: data-driven discovery with large generative models. In *Proc. 41st International Conference on Machine Learning* 34350–34382 (JMLR, 2024).
48. Grossman, R. L. et al. Toward a shared vision for cancer genomic data. *N. Engl. J. Med.* **375**, 1109–1112 (2016).
49. Jupyter. *Jupyter* <https://jupyter.org/> (accessed 23 September 2024).
50. Van Veen, D. et al. Adapted large language models can outperform medical experts in clinical text summarization. *Nat. Med.* **30**, 1134–1142 (2024).
51. Nie, F., Chen, M., Zhang, Z. & Cheng, X. Improving few-shot performance of language models via nearest neighbor calibration. Preprint at <https://arxiv.org/abs/2212.02216> (2022).
52. New embedding models and API updates. *OpenAI* <https://openai.com/index/new-embedding-models-and-api-updates/> (accessed 23 September 2024).
53. Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E. & Singh, S. AutoPrompt: eliciting knowledge from language models with automatically generated prompts. In *Proc. 2020 Conference on Empirical Methods in Natural Language Processing* 4222–4235 (Association for Computational Linguistics, 2020).
54. Vertex AI search. *Google* <https://cloud.google.com/enterprise-search?hl=en> (accessed 23 September 2024).
55. Madaan, A. et al. Self-refine: iterative refinement with self-feedback. *Adv. Neural Inf. Process. Syst.* **36**, 46534–46594 (2023).

Acknowledgements

Z.C. was supported by Japan Society for the Promotion of Science (JSPS) KAKENHI Grant-in-Aid for Scientific Research Number JP24K20778. J.S. was partially supported by National Science Foundation (NSF) awards SCH-2205289, SCH-2014438 and IIS-2034479.

Author contributions

Z.W. and J.S. conceived of and led the overall project. Z.W. and B.D. carried out the experiments and implementations. Z.Y. and Z.C. contributed to the experimental design, conceptualization and dataset construction. Z.W. drafted the paper. J.S. supervised the project and provided a critical review of the paper.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41551-025-01587-2>.

Correspondence and requests for materials should be addressed to Jimeng Sun.

Peer review information *Nature Biomedical Engineering* thanks Chao Yan and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2026

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed
☐	☒ The exact sample size (<i>n</i>) for each experimental group/condition, given as a discrete number and unit of measurement
☐	☒ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☐	☒ The statistical test(s) used AND whether they are one- or two-sided <i>Only common tests should be described solely by name; describe more complex techniques in the Methods section.</i>
☐	☒ A description of all covariates tested
☐	☒ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☐	☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☐	☒ For null hypothesis testing, the test statistic (e.g. <i>F</i> , <i>t</i> , <i>r</i>) with confidence intervals, effect sizes, degrees of freedom and <i>P</i> value noted <i>Give P values as exact values whenever suitable.</i>
☒	☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☐	☒ Estimates of effect sizes (e.g. Cohen's <i>d</i> , Pearson's <i>r</i>), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	Publications were collected from PubMed via the PubMed FTP service, publicly available at https://pubmed.ncbi.nlm.nih.gov/download/ and https://pmc.ncbi.nlm.nih.gov/tools/ftp/ . The patient-level genomics data were collected from cBioPortal, publicly available at https://www.cbioportal.org/datasets .
Data analysis	All experiments were run in Python v3.10. The versions of key software are: anthropic v.0.34.2, boto3 v.1.35.16, openai v.1.44.1, google-generativeai v.0.7.2, google-cloud-aiplatform v.1.65.0, dspy-ai v.2.4.14, langchain v.0.2.16, vLLM v0.9.2, transformers v4.53.2, and docker v.7.1.0 with Python v.3.10. Specifically, we accessed OpenAI's models via OpenAI's platform, the Google models through Vertex AI provided in Google Cloud, and Anthropic's models through AWS's Bedrock APIs. The source code is available at https://github.com/RyanWangZf/BioDSBench .

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

Publications were collected from PubMed via the PubMed FTP service, publicly available at <https://pubmed.ncbi.nlm.nih.gov/download/> and <https://pmc.ncbi.nlm.nih.gov/tools/ftp/>. The patient-level genomics data were collected from cBioPortal, publicly available at <https://www.cbioportal.org/datasets>. The BioDSBench data generated in this study have been deposited in the Hugging Face database under accession code: <https://huggingface.co/datasets/zifeng-ai/BioDSBench>. Source data are provided with this paper.

Research involving human participants, their data, or biological material

Policy information about studies with [human participants or human data](#). See also policy information about [sex, gender \(identity/presentation\), and sexual orientation](#) and [race, ethnicity and racism](#).

Reporting on sex and gender

Reporting on race, ethnicity, or other socially relevant groupings

Population characteristics

Recruitment

Ethics oversight

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☒ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size

Data exclusions

Replication

Randomization

Blinding

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involvement in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern
☒	☐ Plants

Methods

n/a	Involvement in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging

Plants

Seed stocks	n/a
Novel plant genotypes	n/a
Authentication	n/a